decade ranges, and its `.queryset()` method filters the `QuerySet` argument by the decade chosen.

To calculate the range of decades, start with the current year, ignoring the current decade, and then count the 10 decades preceding it. The `.lookups()` method returns a `list` containing a series of `tuples`, with each `tuple` containing the beginning year of decade as a string and another string showing the year range of the decade. The first value gets used as a query parameter in the URL, and although it would be nice for it to be an integer, it has to be a string. When the `.queryset()` method uses it, it must convert it back to an integer to perform the filter.

The `.queryset()` method uses the query parameter argument to affect a filter by date range. The `QuerySet` argument will be the entire set of `Musician` objects, and the filtered version returned needs to be limited to just the chosen decade. This filtering is done by calling `.filter()` on the `QuerySet`, passing in two limiters on the `birth` field. Recall that the `.filter()` method supports the use of suffixes to do comparisons. A decade range is all the dates greater than or equal to January 1st of the first year of the decade, limited by dates less than or equal to December 31st of the last year in the decade. Applying the `__gte` and `__lte` suffixes on the `birth` field in the call to `.filter()` achieves this result.

When a user clicks a link in the filter widget, the same listing page gets reloaded but with a corresponding query parameter set. To save you from having to process the query parameter yourself, the class has a `.value()` convenience method that returns the chosen filter argument. As our filter argument is a number containing the chosen, you convert it back to an integer and then use this to construct the decade date ranges for your `.filter()` call.

The following listing shows the `list_filter` argument to the `MusicianAdmin` class that references the new `DecadeListFilter` and the class code for the filter itself. Modify your admin.py file with the changes shown, and you'll be able to filter your musicians by decade.

Listing 5.4 Custom list filter based on decades

```
# RiffMates/bands/admin.py
...
from datetime import datetime, date
...
class DecadeListFilter(admin.SimpleListFilter):
    title = 'decade born'
    parameter_name = 'decade'

    def lookups(self, request, model_admin):
        result = []

        this_year = datetime.today().year
        this_decade = (this_year // 10) * 10
        start = this_decade - 10
        for year in range(start, start - 100, -10):
```

Import datetime and date for decade calculations.

Inherit from SimpleListFilter to define a new list filter.

Define the list filter's title.

Define the list filter's query parameter name.

Determine the current decade.

Loop backward through the 10 previous decades.